CUSTOMER DEPLOYMENT HANDBOOK

SJC

Manual deployment on Hostinger
A step-by-step upload guide

Deploy the Laravel journal to a new customer domain using Hostinger File Manager for uploads and SSH for one-time application setup.

Prepared: 28 October 2026 | **Method:** manual ZIP upload

CHOOSE THE CORRECT PROCESS
Pages 2-7 install a new website with an empty SQLite database. To transfer an existing journal with its content and accounts, use the separate process on page 8 before running any initialization commands.

What you need	Purpose
Hostinger PHP hosting with SSH and cron	File uploads, PHP execution, setup commands and scheduled tasks.
PHP 8.4.1+ and Composer 2	Required by this application; verify web and command-line PHP separately.
Node.js 22.13+ within 22.x on your computer	Build the browser files before uploading. No Node web server is needed.
A connected domain, SSL and an SMTP mailbox	HTTPS access and real notification delivery.

FOLLOW THE STEPS IN ORDER

02 Steps 1-3: hosting, build and ZIP files
03 Steps 4-5: upload and connect Laravel
04 Steps 6-7: settings, mail and sign-in
05 Steps 8-9: initialize and create the admin
06 Steps 10-11: storage and cron jobs
07 Step 12: verify and troubleshoot
08 Existing-data transfer and references

All account names and domains in commands are examples. Replace uXXXXX and customer-domain.com with the values shown in the customer's hPanel.

STEPS 1-3 / PREPARATION

Prepare the hosting and files

01 Create the website and enable access

Sign in to hpanel.hostinger.com. Add/select the customer's website and connect its domain. Use PHP web hosting with File Manager, SSH and Cron Jobs available.

• In **Websites > Dashboard**, search for **PHP Configuration**, select PHP 8.4 and save. The application requires at least PHP 8.4.1.

• Under PHP extensions, confirm the required modules are active, including PDO SQLite/SQLite, fileinfo, mbstring, intl, GD, BCMath, XML and ZIP. Composer will check its package requirements later.

• Search for **SSH Access**, enable it, and keep the connection command shown there. Do not share the password in the handover document.

Result: the website has PHP hosting, a domain and terminal access. A PHP version change can affect other websites on the same plan; check the plan before applying it.

02 Build the browser files on your computer

Work in a clean handover copy of this project. Open Terminal in that project folder and run:

```
npm ci
npm run build
```

Result: public/build/ contains a manifest and the matching compiled assets, including PDF reader/worker assets. Upload the entire build output together.

03 Prepare two ZIP files

Archive	Contents
journal_app.zip	Application files: app, bootstrap, config, content, database/migrations, database/seeders, resources, routes, scripts, clean storage folders, artisan, composer.json, composer.lock, package files and .env.example. Composer installs vendor on the server.
public_files.zip	The contents of public/, directly at the ZIP root: index.php, .htaccess, build, assets and other public resources. Do not wrap them inside another public/ directory.

KEEP THE HANDOVER COPY CLEAN

Exclude local .env files, local databases, private uploads, logs, release/backup artifacts, node_modules, public/hot, old public/storage links and generated cache files. Preserve the empty runtime folder structure. Include .env.example, .htaccess and other required hidden source files.

Keep both archives private. The application archive belongs outside public_html. File Manager may hide dotfiles until you enable its hidden-file display setting.

STEPS 4-5 / FILE MANAGER

Upload and connect Laravel

04 Upload each ZIP to its correct location

Open **Websites > customer website > Dashboard > File Manager**. Access the website files and move to the parent of public_html. Create journal_app, upload journal_app.zip there, then right-click **Extract**. Put public_files.zip in public_html and extract its contents directly there.

```
domains/customer-domain.com/
+-- journal_app/ app/, bootstrap/, config/, resources/, ...
| +-- artisan
| +-- composer.json / composer.lock / .env.example
+-- public_html/ index.php, .htaccess, build/, assets/, ...
```

Check: journal_app/artisan and public_html/index.php exist at exactly those levels. Move files out of accidental extra extraction folders. Keep .env and the database outside public_html. Remove the uploaded ZIPs after checking extraction.

05 Edit the uploaded production files

A. public_html/index.php: replace its contents with this entry point:

```
<?php
use Illuminate\Foundation\Application;
use Illuminate\Http\Request;
define('LARAVEL_START', microtime(true));
$base = dirname(__DIR__).'/journal_app';
if (file_exists($maintenance = $base.'/storage/framework/maintenance.php')) {
    require $maintenance;
}
require $base.'/vendor/autoload.php';
/** @var Application $app */
$app = require_once $base.'/bootstrap/app.php';
$app->handleRequest(Request::capture());
```

B. journal_app/bootstrap/app.php: change its leading **return Application::configure(...)** to **$app = Application::configure(...)**. Keep all routing, middleware and exception configuration. After its final **})->create();** expression, add:

```
$app->usePublicPath(dirname(__DIR__, 2).'/public_html');
return $app;
```

The existing final expression is **})->create();**; leave it intact. The added public-path binding is required for both web requests and Artisan, so storage links and built assets resolve to public_html.

SUCCESS CHECK
Only public_html is browser-served. It loads the private journal_app application. Apply these edits to the new server copy; do not remove existing middleware or security configuration.

STEPS 6-7 / PRIVATE CONFIGURATION

Configure the new website

06 Create journal_app/.env

Copy .env.example to .env in journal_app. Edit the copy privately. Replace the development database/cache settings with the values below and use the customer's actual account path. Leave APP_KEY empty until step 8.

```
APP_NAME="Customer Journal"
APP_ENV=production
APP_KEY=
APP_DEBUG=false
APP_URL=https://customer-domain.com
APP_FORCE_HTTPS=true
LOG_LEVEL=warning
DB_CONNECTION=sqlite
DB_DATABASE=/home/uXXXXX/domains/customer-domain.com/journal_app/database/database.sqlite
CACHE_STORE=database
SESSION_DRIVER=database
SESSION_DOMAIN=null
SESSION_SECURE_COOKIE=true
QUEUE_CONNECTION=database
FILESYSTEM_DISK=local
MEDIA_DISK=public
SANCTUM_STATEFUL_DOMAINS=customer-domain.com
GOOGLE_OAUTH_ENABLED=false
GOOGLE_ONLY_AUTH=false
SEED_DEMO_CONTENT=false
```

Keep every LOCAL_*_BYPASS_ENABLED flag false. SITEMAP_BASE_URL can retain its ${APP_URL} reference. Keep analytics disabled until its domain/IDs are configured for the customer. A stock .env.example still contains development mail and Redis values; do not use it unchanged.

07 Set up SMTP and the initial sign-in method

For a Hostinger mailbox, set the following private mail values. Replace the example addresses with a real, authorized sender and office recipient:

```
MAIL_MAILER=smtp
MAIL_SCHEME=smtps
MAIL_HOST=smtp.hostinger.com
MAIL_PORT=465
MAIL_USERNAME=notifications@customer-domain.com
MAIL_PASSWORD="your actual mailbox password"
MAIL_FROM_ADDRESS=notifications@customer-domain.com
CONTACT_NOTIFICATION_EMAIL=office@customer-domain.com
ACCOUNT_NOTIFICATION_EMAIL=office@customer-domain.com
```

Use the mail provider's actual settings if the customer uses another provider. Keep secrets in .env, never in a public file or shared PDF. Set the journal's sender name and office routing in its administration settings as well.

SIGN-IN SETUP

Both Google flags are false here so initial setup uses regular email/password login. To enable Google later, create credentials for the new domain and its /auth/google/callback URL. Google-only enforcement requires OAuth to be enabled with valid configured credentials.

STEPS 8-9 / ONE-TIME SSH SETUP

Initialize the application

08 Connect, install dependencies and create tables

On your computer, open Terminal (macOS) or PowerShell (Windows). Paste the SSH command shown in hPanel. Verify the displayed host fingerprint against hPanel, then sign in. Run the following commands in order; replace the account and domain first.

```
cd /home/uXXXXX/domains/customer-domain.com/journal_app
/opt/alt/php84/usr/bin/php -v
mkdir -p storage/framework/cache/data storage/framework/sessions \
  storage/framework/views storage/logs storage/app/private \
  storage/app/public bootstrap/cache database
touch database/database.sqlite
chmod -R u+rwX storage bootstrap/cache database
```

Check: the printed PHP version must be at least 8.4.1. The /opt/alt/php84/usr/bin/php interpreter is used explicitly here because the default SSH php command may differ from the website's PHP selection.

```
/opt/alt/php84/usr/bin/php /usr/local/bin/composer2 install --no-dev --optimize-autoloader
/opt/alt/php84/usr/bin/php /usr/local/bin/composer2 check-platform-reqs --no-dev
/opt/alt/php84/usr/bin/php artisan key:generate --force
/opt/alt/php84/usr/bin/php artisan migrate --force
/opt/alt/php84/usr/bin/php artisan db:seed --force
```

Composer creates vendor/. Migrations create the database tables. Baseline seeding installs roles, permissions and settings; SEED_DEMO_CONTENT=false prevents demo content. Stop and resolve any reported error before running the next command.

FRESH DATABASE ONLY
Run key:generate only for this new, empty installation. If copying an existing database, preserve its original APP_KEY and use page 8. Never run migrate:fresh, db:wipe or demo seeders against an existing journal.

09 Create the first Super Admin privately

```
/opt/alt/php84/usr/bin/php artisan user:create-admin
```

Answer the interactive prompts for name, email and password. The password prompt is private. Use a unique password of at least 12 characters with mixed case, numbers and symbols. This command creates an active, verified Super Admin account.

Result: the application has its dependencies, an initialized SQLite database and a customer administrator. Keep the SQLite file and its parent directory writable by the hosting account; do not use permissions 777.

If /usr/local/bin/composer2 is unavailable, check its location with **command -v composer2** and use that path with the PHP 8.4 interpreter. If SSH, the PHP interpreter or required extensions are unavailable, ask the hosting administrator for assistance.

STEPS 10-11 / STORAGE AND AUTOMATION

Connect uploads and cron jobs

10 Create the public storage link and cache settings

Still in journal_app over SSH, run:

```
/opt/alt/php84/usr/bin/php artisan storage:link
/opt/alt/php84/usr/bin/php artisan optimize
```

Location	What belongs there
journal_app/storage/app/private/	Private manuscripts and controlled workflow/review documents.
journal_app/storage/app/public/	Public images and other files assigned to the public disk.
public_html/storage/	Server-created link to storage/app/public for public file URLs.
journal_app/storage/logs/	Private application logs used when troubleshooting.

Check: storage:link reports success and public_html/storage resolves to the intended public disk. Do not upload a link copied from your computer or move private documents into public_html. If symlinks are blocked, have hosting support/developer assistance configure the public disk mapping.

11 Add two jobs that run every minute

In **Websites > customer website > Dashboard**, search for **Cron Jobs**. Create two **Custom** jobs. Select every minute, hour, day, month and weekday, equivalent to * * * * *. Put only the command in the command field.

Job A - scheduler

```
/opt/alt/php84/usr/bin/php /home/uXXXXX/domains/customer-domain.com/journal_app/artisan schedule:run
```

Job B - guarded mail worker

```
/opt/alt/php84/usr/bin/php /home/uXXXXX/domains/customer-domain.com/journal_app/scripts/hostinger-mail-worker.php
```

Each command is one line. Replace uXXXXX and customer-domain.com. Save both jobs and check their output after the next minute. Do not run a permanent development server or php artisan serve for this deployment.

MAIL WORKER REQUIREMENTS

Configure SMTP before enabling the worker. It requires APP_ENV=production, MAIL_MAILER=smtp and nonempty SMTP username/password. It respects storage/framework/mail-paused and processes the mail,default queues in bounded runs. Inspect an unexpected pause marker before changing it; preserve intentionally held historical queues.

Result: scheduled publication/reminders and queued notifications can run. A successful queue submission or SMTP login alone does not prove inbox delivery.

STEP 12 / LAUNCH CHECKS

Verify the customer website

12 Test the complete experience

Confirm the customer domain points to the hosting account and its SSL certificate is active. Open a fresh browser session and perform these checks:

• **HTTPS and application:** visit https://customer-domain.com/up and the homepage. /up checks application boot; also check real page content and navigation.

• **Administrator:** visit /admin/login and sign in using the Super Admin created in step 9. Review journal name, logo, contact details, sender/office settings and public page content.

• **Files:** upload one appropriate test image/document. Confirm the public image displays and access controls protect private documents.

• **Articles:** once a test article exists, check its PDF download and embedded reader, page navigation and zoom. A fresh database will not include journal articles.

• **Mail:** trigger one intended notification and confirm it arrives in the recipient's inbox. Check spam and cron output if needed.

• **Responsive pages:** inspect desktop and phone navigation, margins, login and article pages.

If something does not work

Symptom	First checks
500 error / blank page	Read journal_app/storage/logs/laravel.log privately. Check PHP 8.4.1+, vendor, .env, APP_KEY, paths and writable directories.
Missing styling or scripts	Check public_html/build/manifest.json and the complete matching assets. Remove a copied public_html/hot file. Confirm publicPath points to public_html.
Missing public images	Check public_html/storage, the recorded file path and permissions. Keep private uploads private.
PDF downloads but reader is blank	Check matching PDF.js assets/worker and JavaScript MIME rules in public_html/.htaccess.
Database error	Confirm the absolute DB_DATABASE path, migrated tables, pdo_sqlite and write access to both file and directory.
Mail does not arrive	Check both cron jobs, SMTP credentials, worker output, mail-paused and failed jobs. Confirm actual delivery.

CLOSE THE HANDOVER

Remove remaining upload archives, keep credentials in the approved private channel, record the two cron jobs, and back up the SQLite database plus matching uploads and configuration. Store a protected backup outside this hosting account and record who owns it.

OPTIONAL / EXISTING JOURNAL DATA

Transfer content without resetting it

Use this process when the customer should receive existing articles, accounts and uploads. It replaces only the fresh-database, key-generation and seeding steps on page 5; dependency installation and runtime-directory setup still apply.

1. Agree the transfer scope. Confirm which accounts, manuscripts, articles, media and settings the customer is authorized to receive.

2. Capture a consistent source backup. Resolve the actual loaded SQLite path. Coordinate writes and workers, create a SQLite backup/snapshot, and collect the matching private/public uploads. Do not copy a changing database file casually.

3. Place the new code and data. Follow the two-folder upload and bootstrap wiring on page 3. Install Composer dependencies and verify writable runtime directories as on page 5. Restore the SQLite backup privately and copy files to their matching configured disks.

4. Preserve the original APP_KEY. Transfer it through a secure channel. Set APP_URL and the new absolute DB_DATABASE path for the destination; review SMTP, journal settings and domain-specific integrations.

5. Skip fresh initialization. Do not generate a new key, create an empty replacement database, or run admin/demo seeders on the copied journal. If code requires schema updates, back up first and run reviewed migrate --force only.

6. Review mail before enabling cron. Keep delivery paused while inspecting copied queued notifications and recipients. Preserve deliberate held queues; do not bulk retry old messages.

7. Verify and release. Recreate/confirm the public storage mapping, clear old Laravel caches and build them again, compare retained records/uploads, then perform the page 7 checks. Keep the rollback copy until acceptance.

DO NOT OVERWRITE AN EXISTING LIVE SITE CASUALLY
The main guide assumes a new destination. For an update or move onto a populated site, take and verify a rollback backup first and plan the change window. Never run database-reset commands as a deployment shortcut.

Record these handover details

Customer domain; Hostinger username; journal_app and public_html paths; active SQLite path; backup location/owner; cron commands; administrator email; SMTP provider/sender. Share passwords, APP_KEY and other secrets separately.

Official instructions and source basis

1. Hostinger: upload and extract archives in File Manager
2. Hostinger: PHP configuration and extensions
3. Hostinger: enable SSH access
4. Hostinger: Composer and the correct PHP interpreter
5. Hostinger: configure cron jobs

Project-specific steps were checked against composer.json, package.json, README, .env.example, public/index.php, bootstrap/app.php, config/filesystems.php, seeders, administrator command and scripts/hostinger-mail-worker.php on 28 October 2026. This document does not certify a new deployment or a fresh inspection of the customer's hosting account.